ASSIGNMENT 13.5

NAME:BANDA PRANEETH
HTNO:2303A51711
BATCH:20

TASK1:

```python
#TASK1
config = {
    "rate": 0.1
}

def calculate_interest(amount, config):
    return amount * config["rate"]
```

```
100.0
```

JUSTIFICATION:

The global variable rate was replaced with a configuration dictionary passed as a parameter. This reduces dependency on external variables and improves modularity. It also makes the function easier to test and maintain.

TASK2:

```python
#TASK2
def get_grade(score):
    if score >= 90:
        return "Excellent"
    if score >= 75:
        return "Very Good"
    if score >= 60:
        return "Good"
    return "Needs Improvement"

score = 78
print(get_grade(score))
```

```
Very Good
```

JUSTIFICATION:

The nested if–else structure was simplified using guard clauses with early returns. This improves readability and reduces code complexity. The logic becomes easier to understand and maintain.

TASK3:

```python
#TASK3
def read_file(filename):
    with open(filename, "r") as f:
        print(f.read())

read_file("data1.txt")
read_file("data2.txt")
```

```
Hello from data1.txt
hello from data2.txt
```

JUSTIFICATION:

Repeated open, read, and close operations were replaced with a reusable function using with open(). The context manager automatically closes the file, improving safety. This follows the DRY principle

and improves code reuse.

TASK4:

```python
#TASK4
users = {
    "admin": "Administrator",
    "guest": "Guest User",
    "editor": "Content Editor",
    "viewer": "Read Only User"
}

name = input("Enter username: ")

if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

```
Enter username: ADMIN
Access Denied
```

JUSTIFICATION:

The list-based search was replaced with a dictionary for user lookup. Dictionaries use hash tables, giving **O(1)** average lookup time instead of **O(n)**. This improves performance and scalability for large datasets.

TASK5:

```python
#TASK5
class EmployeeSalaryCalculator:

    def __init__(self, salary, tax_rate):
        self.salary = salary
        self.tax_rate = tax_rate

    def calculate_tax(self):
        return self.salary * self.tax_rate

    def calculate_net_salary(self):
        tax = self.calculate_tax()
        return self.salary - tax


emp = EmployeeSalaryCalculator(50000, 0.2)
print(emp.calculate_net_salary())
```

```
40000.0
```

JUSTIFICATION

The procedural code was refactored into a class EmployeeSalaryCalculator to follow **object-oriented design principles**.

Encapsulation is achieved by storing salary and tax_rate as class attributes and performing calculations through methods.

This structure improves **code organization, reusability, and maintainability** for larger systems.

TASK6:

```python
#TASK6
n = 999999 // 2
total = n * (n + 1)
print(total)
```

```
249999500000
```

JUSTIFICATION:

The original loop iterates through one million numbers, resulting in **O(n) time complexity**.

The refactored version uses **Python's built-in sum() with comprehension** or a **mathematical formula** to reduce computation.

The formula-based approach achieves **O(1) complexity**, significantly improving performance for large datasets.

TASK7:

```python
#TASK7
def add_item(data, x):
    new_data = data + [x]
    return new_data

data = []
data = add_item(data, 10)
data = add_item(data, 20)

print(data)
```

```
[10, 20]
```

JUSTIFICATION:

The original code modifies a **global mutable list**, creating hidden side effects and reducing predictability.
The refactored version follows a **functional approach** where the function returns a new list instead of modifying global state.
This improves **code reliability, testability, and maintainability**.

TASK8:

```python
#TASK8
def is_length_valid(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)

def validate_password(password):
    if not is_length_valid(password):
        return "Password too short"
    if not has_digit(password):
        return "Must contain digit"
    if not has_uppercase(password):
        return "Must contain uppercase"
    return "Valid Password"

password = input("Enter password: ")
print(validate_password(password))
```

```
Enter password: #diya12345
Must contain uppercase
```

JUSTIFICATION:

The nested validation logic was broken into **separate functions**, each responsible for one rule. This improves **readability and modularity**, making the validation process easier to understand. It also enhances **testability**, since each validation rule can be tested independently.